



JavaFX Database Connection

Login • Register • CRUD • MVC • CSS Styling

4232101 Pemrograman Berorientasi Objek

By: Goklas Henry Agus Panjaitan



Presentation Outline

01 Introduction & JDBC Overview

02 JDBC Architecture & Drivers

03 Database Connection Setup

04 MVC Pattern in JavaFX

05 Model Layer: User & Product

06 DAO Pattern & Database Helper

07 Login & Register (CRUD User)

08 CRUD Product Operations

09 View Layer: FXML + CSS

10 Controllers & Navigation

11 Error Handling & Best Practices

12 Conclusion & References



Introduction: JavaFX + Database

Why connect JavaFX to a Database?

Real-world applications need to persist data beyond a single session. By connecting JavaFX to a relational database (like MySQL), you can build full-stack desktop applications with Login, Register, and CRUD functionalities.



Authentication

Login & register users securely with database-backed credentials



CRUD Operations

Create, Read, Update, Delete data stored in MySQL tables



MVC Architecture

Separate Model, View, Controller for clean, maintainable code



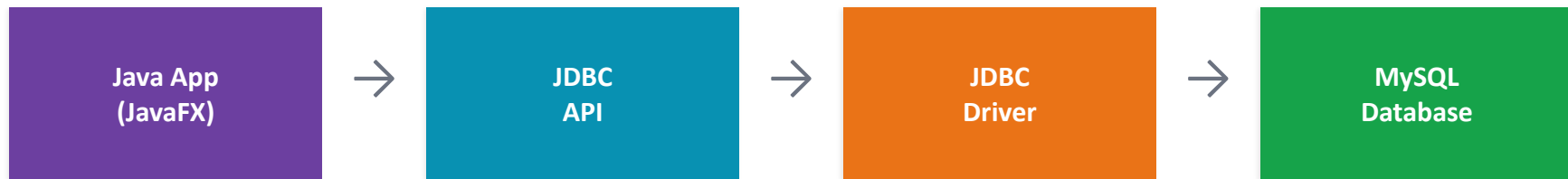
CSS Styled UI

Professional UI with external CSS stylesheets for all views



JDBC Overview

JDBC (Java Database Connectivity) is a standard Java API that allows Java applications to interact with relational databases. It provides methods for querying and updating data.



Key JDBC Classes & Interfaces

DriverManager	Manages database drivers and creates connections
Connection	Represents a session with a specific database
Statement / PreparedStatement	Executes SQL queries (PreparedStatement prevents SQL injection)
ResultSet	Holds the data returned from a SELECT query



JDBC Architecture & Driver Types

JDBC follows a two-tier or three-tier architecture. The driver translates Java calls into database-specific protocol.

Type	Name	Description	Used?
Type 1	JDBC-ODBC Bridge	Uses ODBC driver (deprecated)	No
Type 2	Native-API	Uses native DB client library	Rarely
Type 3	Network Protocol	Middleware translates calls	Sometimes
Type 4	Thin Driver	Pure Java, communicates directly	Yes ☒

For this course, we use Type 4 (Thin Driver) via MySQL Connector/J. Download from: dev.mysql.com/downloads/connector/j/ Add the .jar file to your project's classpath or use Maven dependency.



Database Setup (MySQL)

SQL - Database Schema

```
-- Create Database
CREATE DATABASE javafx_app;
USE javafx_app;

-- Users Table (for Login & Register)
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Products Table (for CRUD)
CREATE TABLE products (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DOUBLE NOT NULL,
    quantity INT DEFAULT 0,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

users table

Stores user credentials.
username is UNIQUE for login.
password should be hashed.

products table

Stores product data for
CRUD operations.
id is auto-incremented.

Data Types

VARCHAR for strings
DOUBLE for prices
TIMESTAMP for dates



DatabaseConnection Class

DatabaseConnection.java (Singleton)

```
public class DatabaseConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/javafx_app";
    private static final String USER = "root";
    private static final String PASS = "";

    private static Connection connection;

    public static Connection getConnection() {
        if (connection == null) {
            try {
                Class.forName("com.mysql.cj.jdbc.Driver");
                connection = DriverManager.getConnection(
                    URL, USER, PASS);
                System.out.println("Connected!");
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
        return connection;
    }

    public static void closeConnection() {
        try {
            if (connection != null && !connection.isClosed())
                connection.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Key Concepts

Singleton Pattern

Only one connection instance is created and reused.

Connection URL

jdbc:mysql://host:port/dbname format.

Class.forName()

Loads the MySQL JDBC driver class at runtime.

DriverManager

Creates the actual connection to the database.

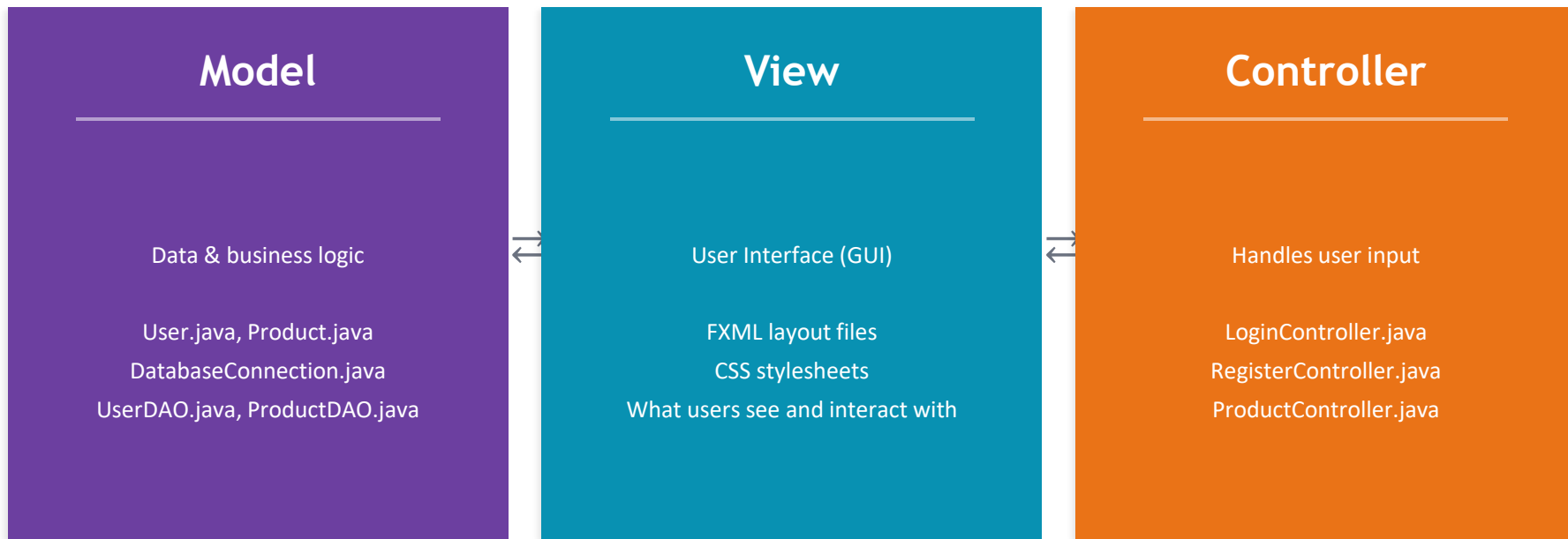
Error Handling

Always wrap in try-catch for SQLException.



MVC Pattern Overview

Model-View-Controller (MVC) separates application concerns into three interconnected components, making code maintainable, testable, and scalable.





MVC Project Structure

Project Directory

```
src/
├── model/
│   ├── User.java           // User entity class
│   ├── Product.java        // Product entity class
│   ├── DatabaseConnection.java
│   ├── UserDAO.java        // User data access
│   └── ProductDAO.java     // Product data access
├── view/
│   ├── login.fxml          // Login screen
│   ├── register.fxml       // Register screen
│   ├── dashboard.fxml      // Product CRUD
│   └── styles.css           // Global CSS
├── controller/
│   ├── LoginController.java
│   ├── RegisterController.java
│   └── ProductController.java
└── MainApp.java            // Entry point
```

Model Layer

Entity classes (POJO) + DAO classes that handle all database operations and business logic.

View Layer

FXML files define the UI layout.
CSS files handle visual styling.
No logic in views.

Controller Layer

Connects View to Model.
Handles button clicks and user input.
Calls DAO methods.

MainApp.java

Application entry point.
Loads the first scene (Login).
Extends Application class.



Model: User.java

User.java - POJO Model

```
public class User {
    private int id;
    private String username;
    private String password;
    private String email;

    // Constructor
    public User(String username, String password,
                String email) {
        this.username = username;
        this.password = password;
        this.email = email;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String u) { username = u; }
    public String getPassword() { return password; }
    public void setPassword(String p) { password = p; }
    public String getEmail() { return email; }
    public void setEmail(String e) { email = e; }
}
```

What is a POJO?

Plain Old Java Object — a simple class with private fields, a constructor, and getters/setters. No framework dependencies.

Encapsulation

Private fields with public getters and setters (OOP)

Maps to Table

Each field corresponds to a column in users table

Reusable

Used across DAO, Controller, and View layers



Model: Product.java

Product.java - POJO Model

```
public class Product {
    private int id;
    private String name;
    private double price;
    private int quantity;
    private String description;

    public Product(String name, double price,
                   int quantity, String description) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
        this.description = description;
    }

    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String n) { name = n; }
    public double getPrice() { return price; }
    public void setPrice(double p) { price = p; }
    public int getQuantity() { return quantity; }
    public void setQuantity(int q) { quantity = q; }
    public String getDescription() { return description; }
    public void setDescription(String d) { description = d; }
}
```

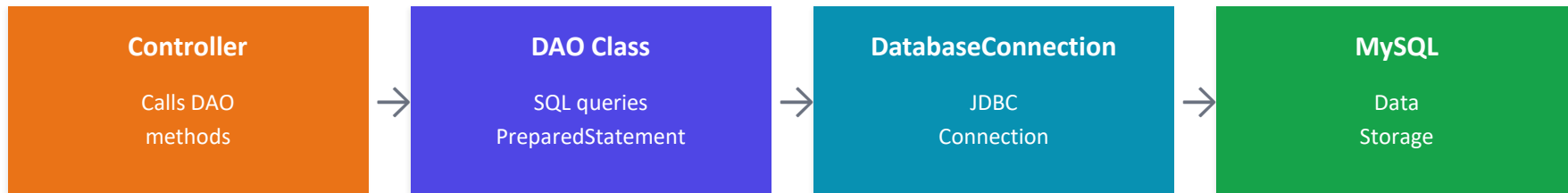
Product POJO mirrors the products table schema. Each field maps to a database column.

Field	Type	DB Column
id	int	INT PK
name	String	VARCHAR(100)
price	double	DOUBLE
quantity	int	INT
description	String	TEXT



DAO Pattern (Data Access Object)

The DAO pattern abstracts and encapsulates all database access. It separates persistence logic from business logic, making your code cleaner and easier to maintain.



Benefits of DAO Pattern

Separation of Concerns

Database logic stays in DAO, not scattered across controllers

Reusability

Same DAO methods can be called from multiple controllers

Testability

Easy to mock DAO classes for unit testing

Maintainability

Change SQL queries in one place without touching UI code



UserDAO: Register Method

UserDAO.java - Register

```
public class UserDAO {
    // REGISTER - Insert new user
    public boolean register(User user) {
        String sql = "INSERT INTO users "
            + "(username, password, email) "
            + "VALUES (?, ?, ?)";
        try {
            Connection conn =
                DatabaseConnection.getConnection();
            PreparedStatement pst =
                conn.prepareStatement(sql);
            pst.setString(1, user.getUsername());
            pst.setString(2, user.getPassword());
            pst.setString(3, user.getEmail());
            int rows = pst.executeUpdate();
            return rows > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
}
```

How It Works

1

PreparedStatement

Uses ? placeholders to prevent SQL injection attacks.

2

setString()

Binds User object values to each ? placeholder safely.

3

executeUpdate()

Runs INSERT and returns the number of affected rows.

4

Return boolean

Returns true if at least 1 row was inserted successfully.



UserDAO: Login Method

UserDAO.java - Login

```
// LOGIN - Verify credentials
public User login(String username, String password) {
    String sql = "SELECT * FROM users "
        + "WHERE username = ? AND password = ?";
    try {
        Connection conn =
            DatabaseConnection.getConnection();
        PreparedStatement pst =
            conn.prepareStatement(sql);
        pst.setString(1, username);
        pst.setString(2, password);
        ResultSet rs = pst.executeQuery();
        if (rs.next()) {
            User user = new User(
                rs.getString("username"),
                rs.getString("password"),
                rs.getString("email"));
            user.setId(rs.getInt("id"));
            return user; // Login success
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null; // Login failed
}
```

Login Flow

1. Receive username & password from LoginController
2. Query database with PreparedStatement (prevents SQL injection)
3. executeQuery() returns a ResultSet
4. If rs.next() is true → user found → return User object
5. If no match → return null → show error in View

Security Note: In production, always hash passwords using BCrypt or similar. Never store plain text passwords!



ProductDAO: Create (INSERT)

ProductDAO.java - CREATE Operation

```
public class ProductDAO {  
    // CREATE - Add new product  
    public boolean addProduct(Product product) {  
        String sql = "INSERT INTO products "  
            + "(name, price, quantity, description) "  
            + "VALUES (?, ?, ?, ?)";  
        try {  
            Connection conn =  
                DatabaseConnection.getConnection();  
            PreparedStatement pst =  
                conn.prepareStatement(sql);  
            pst.setString(1, product.getName());  
            pst.setDouble(2, product.getPrice());  
            pst.setInt(3, product.getQuantity());  
            pst.setString(4, product.getDescription());  
            return pst.executeUpdate() > 0;  
        } catch (SQLException e) {  
            e.printStackTrace();  
            return false;  
        }  
    }  
}
```

C CREATE
INSERT INTO

R READ
SELECT

U UPDATE
UPDATE SET

D DELETE
DELETE FROM



ProductDAO: Read (SELECT)

ProductDAO.java - READ Operation

```
// READ - Get all products
public List<Product> getAllProducts() {
    List<Product> products = new ArrayList<>();
    String sql = "SELECT * FROM products";
    try {
        Connection conn =
            DatabaseConnection.getConnection();
        Statement st = conn.createStatement();
        ResultSet rs = st.executeQuery(sql);
        while (rs.next()) {
            Product p = new Product(
                rs.getString("name"),
                rs.getDouble("price"),
                rs.getInt("quantity"),
                rs.getString("description"));
            p.setId(rs.getInt("id"));
            products.add(p);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return products;
}
```

C CREATE

R READ

U UPDATE

D DELETE



ProductDAO: Update (UPDATE)

ProductDAO.java - UPDATE Operation

```
// UPDATE - Modify existing product
public boolean updateProduct(Product product) {
    String sql = "UPDATE products SET name = ?, "
        + "price = ?, quantity = ?, description = ? "
        + "WHERE id = ?";
    try {
        Connection conn =
            DatabaseConnection.getConnection();
        PreparedStatement pst =
            conn.prepareStatement(sql);
        pst.setString(1, product.getName());
        pst.setDouble(2, product.getPrice());
        pst.setInt(3, product.getQuantity());
        pst.setString(4, product.getDescription());
        pst.setInt(5, product.getId());
        return pst.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```

C CREATE

R READ

U UPDATE

D DELETE



ProductDAO: Delete (DELETE)

ProductDAO.java - DELETE Operation

```
// DELETE - Remove product by ID
public boolean deleteProduct(int id) {
    String sql = "DELETE FROM products WHERE id = ?";
    try {
        Connection conn =
            DatabaseConnection.getConnection();
        PreparedStatement pst =
            conn.prepareStatement(sql);
        pst.setInt(1, id);
        return pst.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
```

C CREATE

R READ

U UPDATE

D DELETE

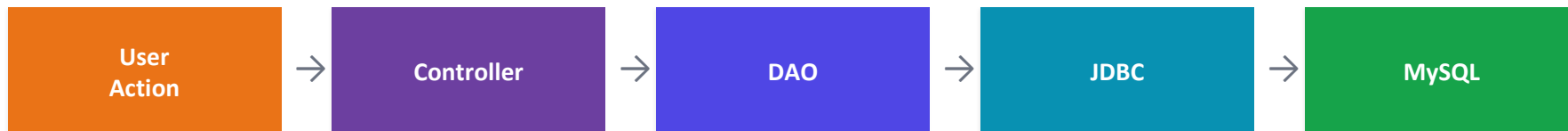
Always use PreparedStatement (not Statement) for any query that includes user input. This prevents SQL injection attacks and is a security best practice.



CRUD Operations Summary

Operation	SQL Command	JDBC Method	Returns	DAO Method
Create	INSERT INTO	executeUpdate()	int (rows)	addProduct()
Read	SELECT	executeQuery()	ResultSet	getAllProducts()
Update	UPDATE SET	executeUpdate()	int (rows)	updateProduct()
Delete	DELETE FROM	executeUpdate()	int (rows)	deleteProduct()

Complete Data Flow





View: Login Screen (FXML)

login.fxml

```
<!-- login.fxml -->
<GridPane fx:controller="controller.LoginController"
    xmlns:fx="http://javafx.com/fxml"
    hgap="10" vgap="12"
    alignment="CENTER" styleClass="login-pane"
    stylesheets="@../view/styles.css">
    <Label text="Login" styleClass="title-label"
        GridPane.columnSpan="2"/>
    <Label text="Username:" styleClass="form-label"
        GridPane.rowIndex="1"/>
    <TextField fx:id="usernameField"
        promptText="Enter username"
        GridPane.columnIndex="1"
        GridPane.rowIndex="1"/>
    <Label text="Password:" styleClass="form-label"
        GridPane.rowIndex="2"/>
    <PasswordField fx:id="passwordField"
        promptText="Enter password"
        GridPane.columnIndex="1"
        GridPane.rowIndex="2"/>
    <HBox spacing="10"
        GridPane.columnIndex="1"
        GridPane.rowIndex="3">
        <Button text="Login"
            onAction="#handleLogin"
            styleClass="btn-primary"/>
        <Hyperlink text="Register"
            onAction="#goToRegister"/>
    </HBox>
    <Label fx:id="messageLabel"
        styleClass="error-label"
        GridPane.columnSpan="2"
        GridPane.rowIndex="4"/>
</GridPane>
```



View: Register Screen (FXML)

register.fxml

```
<!-- register.fxml -->
<GridPane fx:controller="controller.RegisterController"
    xmlns:fx="http://javafx.com/fxml"
    hgap="10" vgap="12"
    alignment="CENTER" styleClass="register-pane"
    stylesheets="@../view/styles.css">
    <Label text="Create Account"
        styleClass="title-label"
        GridPane.columnSpan="2"/>
    <Label text="Username:" styleClass="form-label"
        GridPane.rowIndex="1"/>
    <TextField fx:id="usernameField"
        GridPane.columnIndex="1"
        GridPane.rowIndex="1"/>
    <Label text="Email:" styleClass="form-label"
        GridPane.rowIndex="2"/>
    <TextField fx:id="emailField"
        GridPane.columnIndex="1"
        GridPane.rowIndex="2"/>
    <Label text="Password:" styleClass="form-label"
        GridPane.rowIndex="3"/>
    <PasswordField fx:id="passwordField"
        GridPane.columnIndex="1"
        GridPane.rowIndex="3"/>
    <Button text="Register"
        onAction="#handleRegister"
        styleClass="btn-success"
        GridPane.columnIndex="1"
        GridPane.rowIndex="4"/>
    <Label fx:id="messageLabel"
        GridPane.columnSpan="2"
        GridPane.rowIndex="5"/>
</GridPane>
```



CSS Styling for All Views

styles.css

```
/* styles.css - Global Stylesheet */
.login-pane, .register-pane {
    -fx-background-color: #F8FAFC; -fx-padding: 40;
}
.title-label { -fx-font-size: 24px; -fx-font-weight: bold;
    -fx-text-fill: #1E1B2E; }
.form-label { -fx-font-size: 13px; -fx-text-fill: #4B5563; }
.text-field, .password-field {
    -fx-background-color: white; -fx-border-color: #D1D5DB;
    -fx-border-radius: 6; -fx-background-radius: 6;
    -fx-padding: 8 12; -fx-font-size: 13px;
}
.text-field:focused, .password-field:focused {
    -fx-border-color: #6C3FA0;
    -fx-effect: dropshadow(gaussian, rgba(108,63,160,0.25), 6, 0, 0, 0);
}
.btn-primary { -fx-background-color: #6C3FA0;
    -fx-text-fill: white; -fx-font-weight: bold;
    -fx-padding: 8 24; -fx-background-radius: 6; -fx-cursor: hand; }
.btn-primary:hover { -fx-background-color: #4A2270; }
.btn-success { -fx-background-color: #16A34A;
    -fx-text-fill: white; -fx-font-weight: bold;
    -fx-padding: 8 24; -fx-background-radius: 6; }
.btn-danger { -fx-background-color: #E11D48;
    -fx-text-fill: white; -fx-padding: 6 16;
    -fx-background-radius: 6; }
.error-label { -fx-text-fill: #E11D48; }
```



View: Product Dashboard (FXML)

dashboard.fxml

```
<!-- dashboard.fxml -->
<BorderPane fx:controller="controller.ProductController"
    xmlns:fx="http://javafx.com/fxml"
    stylesheets="@../view/styles.css">
    <top>
        <HBox styleClass="toolbar" spacing="10">
            <Label text="Product Dashboard"
                styleClass="title-label"/>
            <Button text="+ Add Product"
                onAction="#handleAdd" styleClass="btn-success"/>
            <Button text="Logout"
                onAction="#handleLogout" styleClass="btn-danger"/>
        </HBox>
    </top>
    <center>
        <TableView fx:id="productTable">
            <columns>
                <TableColumn fx:id="colId" text="ID"/>
                <TableColumn fx:id="colName" text="Name"/>
                <TableColumn fx:id="colPrice" text="Price"/>
                <TableColumn fx:id="colQty" text="Qty"/>
                <TableColumn fx:id="colDesc" text="Description"/>
                <TableColumn fx:id="colActions" text="Actions"/>
            </columns>
        </TableView>
    </center>
</BorderPane>
```

TableView is a powerful JavaFX control that displays data in rows/columns. Each TableColumn maps to a property in the Product model.



Controller: LoginController

LoginController.java

```
public class LoginController {
    @FXML private TextField usernameField;
    @FXML private PasswordField passwordField;
    @FXML private Label messageLabel;
    private UserDao userDao = new UserDao();

    @FXML
    private void handleLogin(ActionEvent event) {
        String user = usernameField.getText();
        String pass = passwordField.getText();
        if (user.isEmpty() || pass.isEmpty()) {
            messageLabel.setText("Please fill all fields!");
            return;
        }
        User loggedIn = userDao.login(user, pass);
        if (loggedIn != null) {
            // Navigate to Dashboard
            loadScene(event, "dashboard.fxml");
        } else {
            messageLabel.setText("Invalid credentials!");
        }
    }

    @FXML
    private void goToRegister(ActionEvent event) {
        loadScene(event, "register.fxml");
    }
}
```




Controller: RegisterController

RegisterController.java

```
public class RegisterController {
    @FXML private TextField usernameField;
    @FXML private TextField emailField;
    @FXML private PasswordField passwordField;
    @FXML private Label messageLabel;
    private UserDao userDao = new UserDao();

    @FXML
    private void handleRegister(ActionEvent event) {
        String user = usernameField.getText();
        String email = emailField.getText();
        String pass = passwordField.getText();

        if (user.isEmpty() || email.isEmpty()
            || pass.isEmpty()) {
            messageLabel.setText("All fields required!");
            return;
        }
        User newUser = new User(user, pass, email);
        if (userDao.register(newUser)) {
            messageLabel.setText("Success! Please login.");
            messageLabel.setStyle(
                "-fx-text-fill: #16A34A;");
        } else {
            messageLabel.setText(
                "Username already exists!");
        }
    }
}
```



Controller: ProductController

ProductController.java

```
public class ProductController implements Initializable {
    @FXML private TableView<Product> productTable;
    @FXML private TableColumn<Product,String> colName;
    @FXML private TableColumn<Product,Double> colPrice;
    // ... other columns
    private ProductDAO productDAO = new ProductDAO();
    private ObservableList<Product> productList;

    @Override
    public void initialize(URL url, ResourceBundle rb) {
        colName.setCellValueFactory(
            new PropertyValueFactory<>("name"));
        colPrice.setCellValueFactory(
            new PropertyValueFactory<>("price"));
        // ... bind other columns
        loadProducts();
    }

    private void loadProducts() {
        productList = FXCollections.observableArrayList(
            productDAO.getAllProducts());
        productTable.setItems(productList);
    }

    @FXML private void handleAdd(ActionEvent e) {
        // Show dialog, create Product, call DAO
        Product p = new Product(name, price, qty, desc);
        if (productDAO.addProduct(p)) loadProducts();
    }

    @FXML private void handleDelete(Product p) {
        if (productDAO.deleteProduct(p.getId()))
            loadProducts();
    }
}
```



Scene Navigation Utility

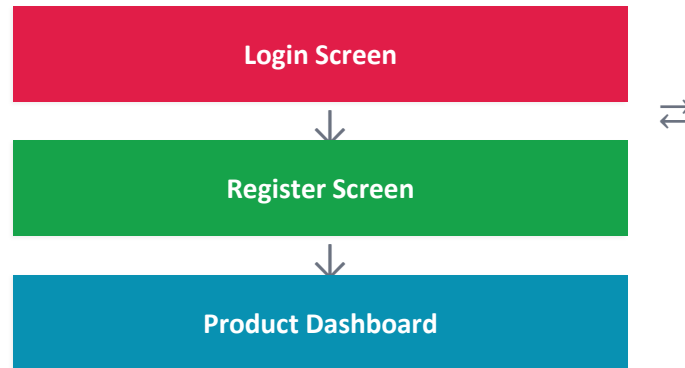
SceneNavigation.java

```
// Utility method - add to a BaseController
// or use in each controller
public void loadScene(ActionEvent event,
                      String fxmlFile) {
    try {
        Parent root = FXMLLoader.load(
            getClass().getResource(
                "/view/" + fxmlFile));
        Stage stage = (Stage)
            ((Node) event.getSource())
                .getScene().getWindow();
        stage.setScene(new Scene(root));
        stage.show();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

MainApp.java Entry Point

```
public class MainApp extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        Parent root = FXMLLoader.load(
            getClass().getResource("/view/login.fxml"));
        stage.setTitle("JavaFX App");
        stage.setScene(new Scene(root, 800, 600));
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

Application Flow





Error Handling & Validation

Input Validation

Check for empty fields before calling DAO.
Validate email format and password length.
Show user-friendly error messages.

Duplicate Username

Handle `SQLIntegrityConstraintViolation`
for `UNIQUE` constraint violations.
Show 'Username already exists' message.

User Feedback

Use Labels to show success/error messages.
Color-code: green for success, red for errors.
Clear messages when user starts typing.

SQLException Handling

Wrap all JDBC operations in try-catch.
Log errors with `e.printStackTrace()`.
Return null or false on failure.

Connection Null Check

Always verify connection is not null
before executing queries.
Implement connection pooling for production.

Resource Cleanup

Close `ResultSet`, `PreparedStatement`
when done. Use try-with-resources
for automatic cleanup.

Conclusion

- ✓ JDBC provides the standard API for connecting JavaFX applications to relational databases like MySQL.
- ✓ The DAO Pattern encapsulates all SQL operations, keeping controllers clean and focused on UI logic.
- ✓ MVC architecture separates Model (data), View (FXML/CSS), and Controller (logic) for maintainable code.
- ✓ PreparedStatement prevents SQL injection and is essential for secure database operations.
- ✓ CSS styling enables professional, polished user interfaces with clean separation from Java code.
- ✓ Login, Register, and CRUD form the foundation of most real-world database-driven applications.

Build • Connect • Create • Persist



References

- [1] B. Eckel, Thinking in Java, 4th ed. Prentice Hall, 2006.
- [2] H. Schildt, Java: The Complete Reference, 11th ed. Oracle Press, 2019.
- [3] R. Pecinovsky, OOP – Learn Object Oriented Thinking & Programming. Unikov, 2013.
- [4] K. Sierra and B. Bates, Head First Java, 2nd ed. O'Reilly Media, 2005.
- [5] S. Chin, J. Vos, J. Weaver, The Definitive Guide to Modern Java Clients with JavaFX. Apress, 2019.
- [6] Oracle Corporation, "JavaFX Documentation," docs.oracle.com/javase/8/javafx/.
- [7] Oracle Corporation, "JDBC Tutorial," docs.oracle.com/javase/tutorial/jdbc/.
- [8] MySQL, "Connector/J Developer Guide," dev.mysql.com/doc/connector-j/en/.
- [9] Baeldung, "JavaFX Tutorial," baeldung.com/javafx. [Online].
- [10] J. Jenkov, "Java JDBC Tutorial," jenkov.com/java-jdbc/index.html. [Online].